WEB APPLICATION DEVELOPMENT WITH NODE.JS

GV: TRẦN THANH TUẤN

04. EXPRESS – MYSQL2

EXPRESS

- Fast, unopinionated, minimalist web framework for NODEJS

```
npm install express
```

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
	res.send('Hello World!')
})

app.listen(port, () => {
	console.log(`App is running on port ${port}`)
})
```

EXPRESS

- **Routing**
  - Routing refers to determining how an application responds to a client request to a particular endpoint, which is a **URI** *(or path)* and a specific **HTTP request method** *(GET, POST, PUT, PATCH, DELETE…).*
  - Each route can have one or more handler functions, which are executed when the route is matched.

```
app.METHOD(PATH, HANDLER)
```

An instance of express

A HTTP request method, in lowercase

The function executed when the route is matched

A path on the server, can be strings, string patterns, or regular expressions

EXPRESS

- **Route path examples**

```
app.get('/', (req, res) => {
        res.send('root')
})
```

will match requests to the root route, /

```
app.get('/about', (req, res) => {
        res.send('about')
})
```

will match requests to /about

EXPRESS

- **Route path examples**

```
app.get('/ab?cd', (req, res) => {
        res.send('/ab?cd')
})
```

will match /acd and /abcd

```
app.get('/ab+cd', (req, res) => {
        res.send('/ab+cd')
})
```

will match requests to /abcd, /abbcd, /abbbcd, /ab...bcd

```
app.get('/ab*cd', (req, res) => {
        res.send('/ab*cd')
})
```

will match requests to /abcd, /abxcd, /abRANDOMcd, /ab123cd, /ab...cd

```
app.get('/ab(cd)?e', (req, res) => {
        res.send('/ab(cd)?e')
})
```

will match requests to /abe and /abcde

EXPRESS

- **Route path examples**

```
app.get('/a/', (req, res) => {
        res.send('/a/')
})
```

will match anything with an "a" in it

```
app.get('/.*fly$', (req, res) => {
        res.send('/.*fly$')
})
```

will match /butterfly and /dragonfly, but not /butterflyman, /dragonflyman …

EXPRESS

- **Route parameters** are named URL segments that are used to capture the values specified at their position in the URL. The captured values are populated in the req.params object, with the name of the route parameter specified in the path as their respective keys.

Route path: /users/:userId/books/:bookId
Request URL: http://localhost:3000/users/34/books/8989
req.params: { "userId": "34", "bookId": "8989" }

The name of route parameters must be made up of “word characters” ([A-Za-z0-9_]).

EXPRESS

- **Route parameters**
  - The hyphen (-) and the dot (.) are interpreted literally, they can be used along with route parameters for useful purposes

Route path: /flights/:from-:to
Request URL: http://localhost:3000/flights/LAX-SFO
req.params: { "from": "LAX", "to": "SFO" }

Route path: /plantae/:genus.:species
Request URL: http://localhost:3000/plantae/Prunus.persica
req.params: { "genus": "Prunus", "species": "persica" }

EXPRESS

- Route handlers
  - Can provide multiple callback functions to handle a request
  - Route handler can be in the form of a function, an array of functions, or combinations of both

```
app.get('/example/a', (req, res) => {
        res.send('Hello from A!')
})
```

A single callback function can handle a route

EXPRESS

- Route handlers

```
app.get('/example/b', (req, res, next) => {
        console.log('the response will be sent by the next function ...’)
        next()
}, (req, res) => {
        res.send('Hello from B!')
})
```

More than one callback function can handle a route

EXPRESS

- Route handlers

```
const cb0 = function (req, res, next) {
        console.log('CB0')
        next()
}
```

```
const cb1 = function (req, res, next) {
        console.log('CB1')
        next()
}
```

```
const cb2 = function (req, res) {
        res.send('Hello from C!')
}
```

```
app.get('/example/c', [cb0, cb1, cb2])
```

An array of callback functions can handle a route

```
const cb0 = function (req, res, next) {
        console.log('CB0')
        next()
}
```

```
const cb1 = function (req, res, next) {
        console.log('CB1')
        next()
}
```

```
app.get('/example/d', [cb0, cb1], (req, res, next) => {
        console.log('the response will be sent by the next function ...')
        next()
}, (req, res) => {
        res.send('Hello from D!')
})
```

A combination of independent functions and arrays of functions can handle a route

EXPRESS

- **Response methods**
  - The methods on the response object **(res)** in the following table can send a response to the client, and terminate the request-response cycle.
  - If none of these methods are called from a route handler, the client request will be left hanging.

METHOD	DESCRIPTION
res.download()	Prompt a file to be downloaded.
res.end()	End the response process.
res.json()	Send a JSON response.
res.jsonp()	Send a JSON response with JSONP support.
res.redirect()	Redirect a request.
res.render()	Render a view template.
res.send()	Send a response of various types.
res.sendFile()	Send a file as an octet stream.
res.sendStatus()	Set the response status code and send its string representation as the response body.

EXPRESS

- **Request properties**
  - The properties on the request object **(req)** in the following table.

METHOD	DESCRIPTION
req.app	Holds a reference to the instance of the Express application that is using the middleware
req.baseUrl	The URL path on which a router instance was mounted
req.body	Contains key-value pairs of data submitted in the request body. By default, it is undefined, and is populated when you use body-parsing middleware such as **body-parser** and **multer**
req.cookies	When using cookie-parser middleware, this property is an object that contains cookies sent by the request. If the request contains no cookies, it defaults to {}
req.host	Contains the host derived from the Host HTTP header
req.hostname	Contains the hostname derived from the Host HTTP header
req.ip	Contains the remote IP address of the request
req.method	Contains a string corresponding to the HTTP method of the request: GET, POST, PUT, …
req.originalUrl	This property is much like req.url;

EXPRESS

- **Request properties**
  - The properties on the request object **(req)** in the following table.

METHOD	DESCRIPTION
req.params	This property is an object containing properties mapped to the named route “parameters”
req.path	Contains the path part of the request URL
req.protocol	Contains the request protocol string: either **http** or (for TLS requests) **https**
req.query	This property is an object containing a property for each query string parameter in the route. When **query parser** is set to disabled, it is an empty object {}, otherwise it is the result of the configured query parser.

https://expressjs.com/en/5x/api.html#req.app

EXPRESS

- **app.route()**
  - Create chainable route handlers for a route path

```
app.route('/book')
        .get((req, res) => {
                res.send('Get a random book')
        })
        .post((req, res) => {
                res.send('Add a book')
        })
        .put((req, res) => {
                res.send('Update the book')
        })
```

EXPRESS

- **express.Router**: create modular, mountable route handlers.
  - A Router instance is a complete middleware and routing system; for this reason, it is often referred to as a **“mini-app”**

```
const express = require('express')
const router = express.Router()

// middleware that is specific to this router
const timeLog = (req, res, next) => {
        console.log('Time: ', Date.now())
        next()
}
router.use(timeLog)

// define the home page route
router.get('/', (req, res) => {
        res.send('Birds home page')
})

// define the about route
router.get('/about', (req, res) => {
        res.send('About birds')
})

module.exports = router
```

The app will now be able to handle requests to /birds and /birds/about, as well as call the timeLog middleware function that is specific to the route.

```
const birds = require('./birds')

// ...
app.use('/birds', birds)
```

EXPRESS

- **Serving static files**
  - To serve static files such as **images, CSS files, and JavaScript files**, use the **express.static** built-in middleware function in Express.

```
express.static(root, [options])
```

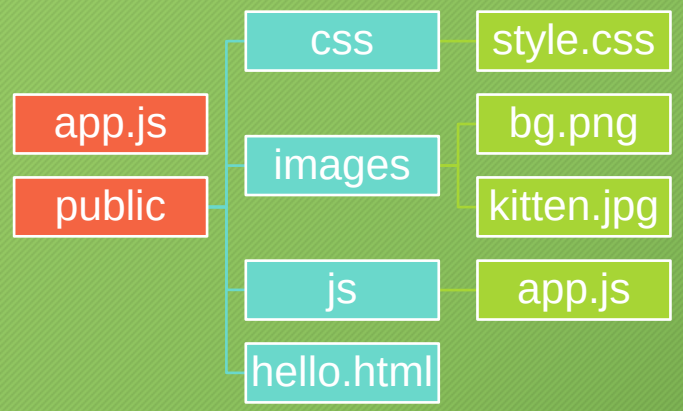
the root directory from which to serve static assets

https://expressjs.com/en/5x/api.html#express.static

EXPRESS

- **Serving static files**
  - Examples
    - Serve images, CSS files, and JavaScript files in a directory named public

```
app.use(express.static('public'))
```

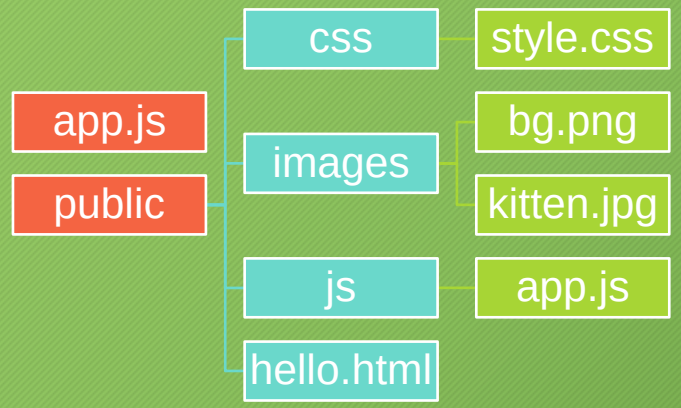


http://localhost:3000/images/kitten.jpg
http://localhost:3000/css/style.css
http://localhost:3000/js/app.js
http://localhost:3000/images/bg.png
http://localhost:3000/hello.html

EXPRESS

- **Serving static files**
  - Examples
    - To create a virtual path prefix *(where the path does not actually exist in the file system)* for files that are served by the **express.static** function, specify a mount path for the static directory

```
app.use('/static' , express.static('public'))
```

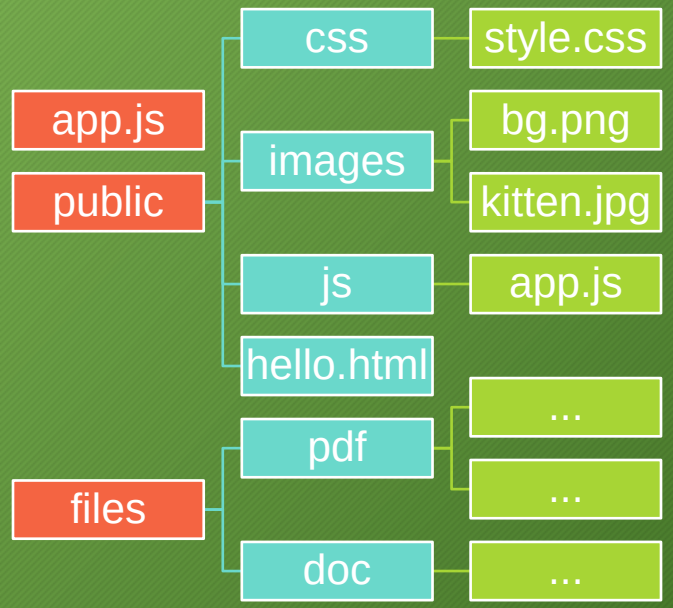


http://localhost:3000/static/images/kitten.jpg
http://localhost:3000/static/css/style.css
http://localhost:3000/static/js/app.js
http://localhost:3000/static/images/bg.png
http://localhost:3000/static/hello.html

EXPRESS

- **Serving static files**
  - Examples
    - To use **multiple** static assets directories, call the **express.static** middleware function multiple times

```
app.use(express.static('public'))
app.use(express.static('files'))
```



MySQL2

- MySQL client for Node.js with focus on performance. Supports prepared statements, non-utf8 encodings, binary log protocol, compression, SSL much more.
- MySQL2 project is a continuation of **MySQL-Native**
- MySQL2 is mostly API compatible with **Node MySQL** and supports majority of features

https://github.com/mysqljs/mysql

MySQL2

- **Installation**

```
npm install mysql2
```

- **createConnection(connectionUri: string)**

```
import mysql from 'mysql2/promise';

try {
	const connection = await mysql.createConnection(
		'mysql://root:password@localhost:3306/test'
	);
} catch (err) {
	console.log(err);
}
```

promise

```
const mysql = require('mysql2');

const connection = mysql.createConnection(
	'mysql://root:password@localhost:3306/test'
);
connection.addListener('error', (err) => {
	console.log(err);
});
```

callback

MySQL2

- **createConnection(config: ConnectionOptions)**

```
import mysql from 'mysql2/promise';

try {
	const connection = await mysql.createConnection({
		host: 'localhost',
		user: 'root',
		database: 'test',
		port: 3306,
		password: 'password',
	});
} catch (err) {
	console.log(err);
}
```

promise

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
	host: 'localhost',
	user: 'root',
	database: 'test',
	port: 3306,
	password: 'password',
});

connection.addListener('error', (err) => {
	console.log(err);
});
```

callback

MySQL2

- **createPool(connectionUri: string)**

```
import mysql from 'mysql2/promise';

try {
	const pool = mysql.createPool('mysql://root:password@localhost:3306/test');
	const connection = await pool.getConnection();
	// ... some query

	connection.release();
} catch (err) {
	console.log(err);
}
```

promise

callback

```
const mysql = require('mysql2');

const pool = mysql.createPool('mysql://root:password@localhost:3306/test');

pool.getConnection(function (err, connection) {
	if (err instanceof Error) {
		console.log(err);
		return;
	}

	// ... some query

	connection.release();
});
```

MySQL2

- **createPool(config: PoolOptions)**

```
import mysql from 'mysql2/promise';

try {
	const pool = mysql.createPool({
		host: 'localhost',
		user: 'root',
		database: 'test',
		port: 3306,
		password: 'password',
	});
	const connection = await pool.getConnection();
	// ... some query

	connection.release();
} catch (err) {
	console.log(err);
}
```

promise

callback

```
const mysql = require('mysql2');

const pool = mysql.createPool({
	host: 'localhost',
	user: 'root',
	database: 'test',
	port: 3306,
	password: 'password',
});

pool.getConnection(function (err, connection) {
	if (err instanceof Error) {
		console.log(err);
		return;
	}

	// ... some query

	connection.release();
});
```

MySQL2

- **Simple Queries**
  - SELECT

promise

```
try {
	const sql = "SELECT * FROM `users` WHERE `name` = 'Page' AND `age` > 45";
	const [rows, fields] = await connection.query(sql);
	console.log(rows);
	console.log(fields);
} catch (err) {
	console.log(err);
}
```

```
const sql = "SELECT * FROM `users` WHERE `name` = 'Page' AND `age` > 45";
connection.query(sql, (err, rows, fields) => {
	if (err instanceof Error) {
		console.log(err);
		return;
	}
	console.log(rows);
	console.log(fields);
});
```

callback

MySQL2

- **Simple Queries**
  - INSERT

promise

```
try {
	const sql = "INSERT INTO `users`(`name`, `age`) VALUES ('Josh', 19), ('Page', 45)";
	const [result, fields] = await connection.query(sql);
	console.log(result);
	console.log(fields);
} catch (err) {
	console.log(err);
}
```

```
const sql = "INSERT INTO `users`(`name`, `age`) VALUES ('Josh', 19), ('Page', 45)";
connection.query(sql, (err, result, fields) => {
	if (err instanceof Error) {
		console.log(err);
		return;
	}
	console.log(result);
	console.log(fields);
});
```

callback

MySQL2

- **Simple Queries**
  - UPDATE

promise

```
try {
	const sql = "UPDATE `users` SET `age` = 20 WHERE `name` = 'Josh'";
	const [result, fields] = await connection.query(sql);
	console.log(result);
	console.log(fields);
} catch (err) {
	console.log(err);
}
```

```
const sql = "UPDATE `users` SET `age` = 20 WHERE `name` = 'Josh'";
connection.query(sql, (err, result, fields) => {
	if (err instanceof Error) {
		console.log(err);
		return;
	}
	console.log(result);
	console.log(fields);
});
```

callback

MySQL2

- **Simple Queries**
  - DELETE

promise

```
try {
	const sql = "DELETE FROM `users` WHERE `name` = 'Page'";
	const [result, fields] = await connection.query(sql);
	console.log(result);
	console.log(fields);
} catch (err) {
	console.log(err);
}
```

```
const sql = "DELETE FROM `users` WHERE `name` = 'Page'";
connection.query(sql, (err, result, fields) => {
	if (err instanceof Error) {
		console.log(err);
		return;
	}
	console.log(result);
	console.log(fields);
});
```

callback

MySQL2

- **Prepared Statements**
  - SELECT

promise

```
try {
	const sql = 'SELECT * FROM `users` WHERE `name` = ? AND `age` > ?';
	const values = ['Page', 45];
	const [rows, fields] = await connection.execute(sql, values);

	console.log(rows);
	console.log(fields);
} catch (err) {
	console.log(err);
}
```

```
const sql = 'SELECT * FROM `users` WHERE `name` = ? AND `age` > ?';
const values = ['Page', 45];
connection.execute(sql, values, (err, rows, fields) => {
	if (err instanceof Error) {
		console.log(err);
		return;
	}
	console.log(rows);
	console.log(fields);
});
```

callback

MySQL2

- **Prepared Statements**
  - INSERT

promise

```
try {
	const sql = 'INSERT INTO `users`(`name`, `age`) VALUES (?, ?), (?,?)';
	const values = ['Josh', 19, 'Page', 45];
	const [result, fields] = await connection.execute(sql, values);

	console.log(result);
	console.log(fields);
} catch (err) {
	console.log(err);
}
```

```
const sql = 'INSERT INTO `users`(`name`, `age`) VALUES (?, ?), (?,?)';
const values = ['Josh', 19, 'Page', 45];
connection.execute(sql, values, (err, result, fields) => {
	if (err instanceof Error) {
		console.log(err);
		return;
	}
	console.log(result);
	console.log(fields);
});
```

callback

MySQL2

- **Prepared Statements**
  - UPDATE

promise

```
try {
	const sql = 'UPDATE `users` SET `age` = ? WHERE `name` = ?';
	const values = [20, 'Josh'];
	const [result, fields] = await connection.execute(sql, values);

	console.log(result);
	console.log(fields);
} catch (err) {
	console.log(err);
}
```

```
const sql = 'UPDATE `users` SET `age` = ? WHERE `name` = ?';
const values = [20, 'Josh'];
connection.execute(sql, values, (err, result, fields) => {
	if (err instanceof Error) {
		console.log(err);
		return;
	}
	console.log(result);
	console.log(fields);
});
```

callback

MySQL2

- **Prepared Statements**
  - DELETE

promise

```
try {
	const sql = 'DELETE FROM `users` WHERE `name` = ?';
	const values = ['Page'];
	const [result, fields] = await connection.execute(sql, values);

	console.log(result);
	console.log(fields);
} catch (err) {
	console.log(err);
}
```

```
const sql = 'DELETE FROM `users` WHERE `name` = ?';
const values = ['Page'];
connection.execute(sql, values, (err, result, fields) => {
	if (err instanceof Error) {
		console.log(err);
		return;
	}
	console.log(result);
	console.log(fields);
});
```

callback

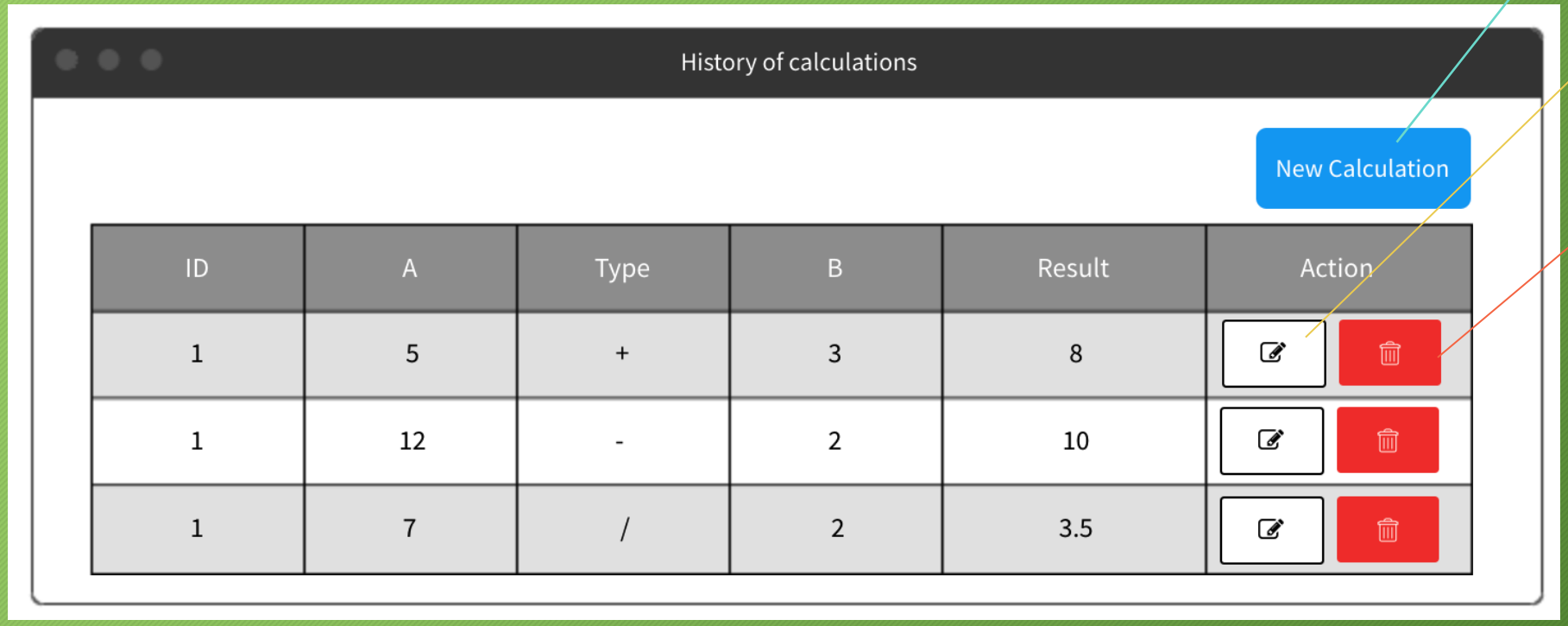
Exercise

- Create database **exercises_db**
- Create table **calculations**

Field name	Type	Note
id	integer	Primary key, Auto Increment
type	varchar(1)	+, -, *, /
a	interger	
b	interger	
result	float	

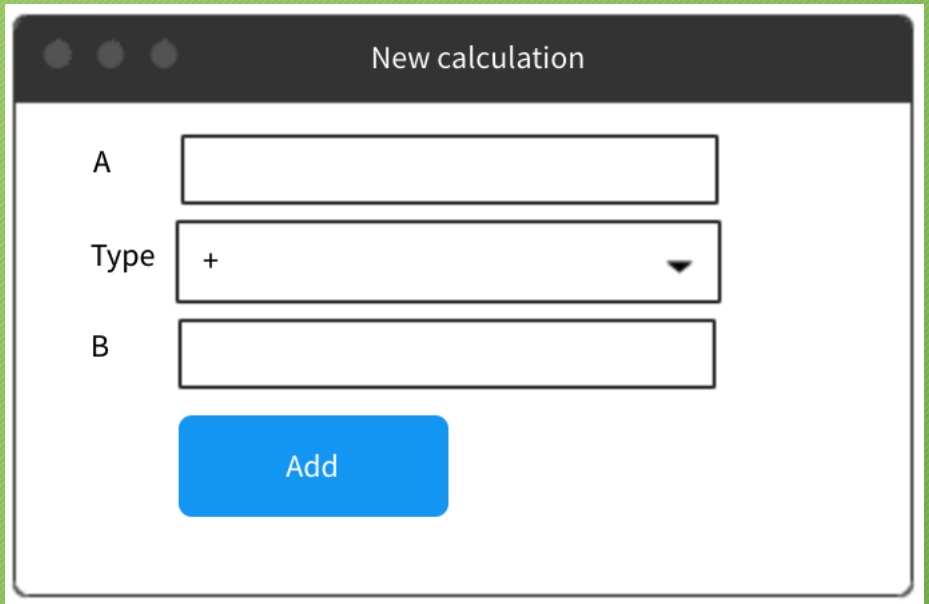
Exercise

- Build a web application
  - List history of calculations



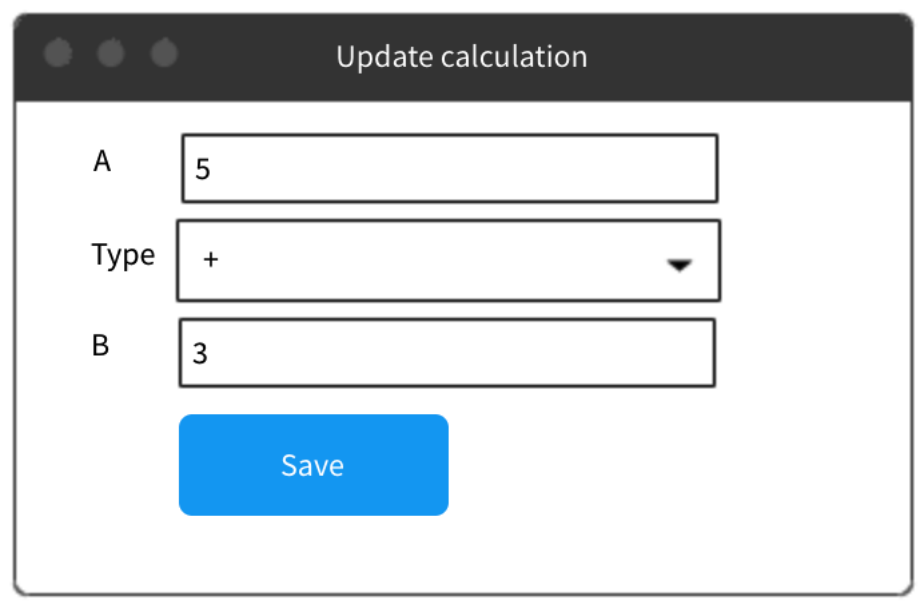
Exercise

- Build a web application
  - Add new calculation **(1)**



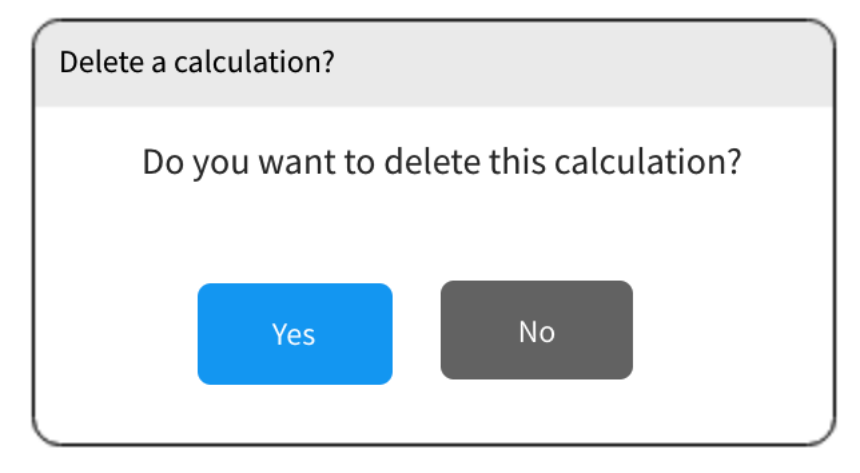
Exercise

- Build a web application
  - Update a calculation **(2)**



Exercise

- Build a web application
  - Delete a calculation **(3)**



Exercise

- Use Express, MySQL2, ***EJS***
- Routes
  - **GET** /calculations
  - **GET** /calculations/add-new
  - **POST** /calculations/add-new
  - **GET** /calculations/:id/update
  - **POST** /calculations/:id/update
  - **GET** /calculations/:id/delete

Thank you!